

SMART GRID MANAGEMENT SYSTEM SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

PHASE 0: Vision and Product Framing.

Problem Statement

- Difficulties in efficiently distributing energy (electricity) after several failures (system failures)
- Suffer from prolonged blackouts (absence of electricity)
- Incorrect shedding of load. Loads types include hospitals, offices, schools etc

Existing Solutions

Although Cameroon is trying to implement distributed generation (Having different power plants for backup purposes), there are no current solutions to the problem encountered by the traditional grid system. Also, some experiments have been carried out by a group of researchers at the level of the Kumba grid system, where they reported the problem faced by the current system in Kumba and they effectively approached the problem. More details about this experiments can be found in the their article in [Distributed Generation and Optimization of Smart Grid Systems: Case Study of Kumba in Cameroon](#)

Value Proposition

A smart grid management system that runs simulations to see how electricity is optimally distributed under different scenarios such as failures, peak demand, micro grids isolation.

Target Users

- Grid operators
- Teachers or students that run simulations
- Energy regulators

Success metrics

The system must be able to simulate effectively:

- Blackouts
- Transmission line failures
- Peak demands
- Micro grid isolation
- Calculating cost, demand, supply etc.
- Efficiently propose distribution plan
- Warnings of current implemented system topology

PHASE 1: Requirements Engineering

Stakeholder Analysis

- Grid Operators or Planners
Uses the system to plan distribution. They need it for easy visualization, scenario simulation and allocation recommendations.
- Power Engineers or Researchers
Validates algorithms and scenarios. They can have access to simulation metrics.
- Students or Trainees
Learn about smart grid behavior. They might need an interactive frontend, intuitive user interface with predefined scenarios.
- Project Team
Build and maintain the system. They need to define clear requirements, structured workflow, and manageable scope.
- Instructors or Evaluators
Access correctness and completeness. They need a clear demonstration of the simulation, logic, dual interface (CLI and Web), algorithms functioning correctly.

Functional Requirements (What the system should do)

1. Scenario Management
 - ❖ Load predefined scenarios from JSON
 - ❖ Allow user to create/edit scenarios via frontend
 - ❖ Save/export scenarios
2. Grid Modeling
 - Represents plants, loads and lines as nodes and edges
 - Support different plant types (hydro, solar, thermal)
 - Support line capacities, losses and priorities
3. Simulation and Optimization
 - Compute power allocation under given scenarios
 - Handle failure scenarios such as line or plant outage, peak demand etc.
 - Use graph algorithms, priority queues, segment trees, union-find as needed
4. Visualization and Analytics
 - Display grid topology graphically

- Show allocation results that is which plant supplies which load
- Display metrics total supply is demand, unserved loads, cost, line loads.

5. Dual Interface

- CLI: read JSON input, output allocation and metrics
- Web: Interactive simulation with visualization.

Non-Functional Requirements (System qualities)

- I. Usability
Frontend must be intuitive for non-experts users (drag and drop, simple menus)
- II. Performance
Simulation should run quickly for moderate grid size eg. ≤ 50 nodes.
- III. Scalability
Can extend to more node lines or scenarios in future
- IV. Portability
Web app runs in browser, CLI runs on any system with Python
- V. Maintainability
Well Structured code separation of frontend and backend, reusable algorithms.
- VI. Accuracy
Simulation outputs reflects correct allocation logic

Constraints

No real-time sensor or IoT integration: Only scenario-based simulation

Limited Dataset: Inputs come from JSON or frontend forms

Time/resource constraints: Implement MVP first, advanced features like drag and drop analytics are optional

Tech Stack: Python(backend), Reactjs(Frontend), JSON for data interchange